Descriptive Statistics with R

Jinsong Chen

Table of contents

1 Questions, Records and a First Summary **3**
1.1 State what the number describes . . . 3
1.2 R, RStudio and a saved script . . . 4
1.3 Values, vectors and function arguments . . . 4
1.4 Base R and a small selection of tidyverse tools . . . 5
1.5 Where Quarto fits . . . 5

2 Select Observations and Preserve the Measurement **6**
2.1 Rows, columns and missing values . . . 6
2.2 Inspect the observations needed for a summary . . . 7
2.3 One selection in base R and dplyr . . . 8
2.4 Transform units without changing the measurement . . . 9

3 Describe a Distribution **9**
3.1 Center and variation answer different questions . . . 9
3.2 Quartiles, IQR and the numerical convention . . . 10
3.3 Read the shape alongside the numbers . . . 11

4 Compare Groups and Conditional Percentages **12**
4.1 Read the group count beside its mean . . . 12
4.2 Choose the denominator of a conditional percentage . . . 14

5 Describe Quantitative Relationships **15**
5.1 Covariance retains units; correlation standardizes them . . . 15
5.2 A small correlation can miss a strong curved pattern . . . 17

6 Describe Paired Differences **18**
6.1 Align the measurements before subtracting . . . 18
6.2 Describe the distribution of differences . . . 20

7 Report the Descriptive Answer **21**
7.1 Connect the number to the observations . . . 21
7.2 Combine a descriptive result into a Quarto report . . . 22
7.3 Summary and Reading Bridge . . . 23

Exercises **24**
Exercise 1: Count the right observations (Foundational) 25
Exercise 2: Center, spread and a changed observation (Foundational) 25
Exercise 3: A missing score is not zero (Foundational) 25
Exercise 4: Choose the group denominator (Foundational) 25
Exercise 5: Read a relationship before evaluating it (Integrative) 25
Exercise 6: Report paired differences (Integrative) 26

Appendix A: Supporting Table Operations **26**
Return to a long layout when it is useful 26
Add information from a lookup table 27
Files and a short reproducible report 28

References **28**

Chapter box

Central question: What do the included observations show about a distribution, a group comparison or a relationship, and how can we reproduce and explain that descriptive answer with simple R?
Focal concepts: Observations and denominators; mean, median, variance, SD, quartiles and IQR; graphical distributions; grouped summaries and conditional percentages; covariance, correlation and paired differences; elementary R objects, the base R/tidyverse approach and integrated reporting.
Main evidence: Small score tables introduce the calculations. Built-in `airquality` provides 153 historical days, including 116 recorded ozone values; `mtcars` supplies 32 car-model records; `sleep` provides paired measurements for ten patients.
Scope: Chapter 2 supplies descriptive foundations and Chapter 1 the reading approach. No previous R programming is required. Statistics organizes the examples; the lab supplies setup and operating practice. Chapter 6 adds uncertainty under stated sampling or model assumptions, and Chapter 7 connects variation and covariance to regression.

1 Questions, Records and a First Summary

1.1 State what the number describes

A descriptive result answers a question about specified observations. Before calculating, name the row unit, variable, units and included records. For three recorded scores of 50, 60 and 70 points, the arithmetic mean is

$$\bar{x} = \frac{50 + 60 + 70}{3} = 60 \text{ points.}$$

R can reproduce that calculation with two short lines:

```
scores <- c(50, 60, 70)
mean(scores)
#> [1] 60
```

`c()` collects the values into a vector; `<-` gives it the name `scores`. `mean(scores)` asks a function to calculate its mean. The returned 60 is a summary of these three observations. A population interpretation needs an additional question and justification. There is no need for a p-value to know the mean of the recorded values.

Chapter 2 explained what the summary means. Here the formula, the R command and the interpretation describe the same calculation. We use that sequence throughout the chapter: question, observations, calculation and answer.

1.2 R, RStudio and a saved script

R is a language and platform for statistical computing: it stores data, calculates statistics and draws displays. Statistical programming begins with short instructions such as the two-line mean calculation above. An **object** is a named value, vector, table or other result stored by R. A **function** performs an operation on supplied inputs.

RStudio is an integrated development environment (IDE). Its Source editor holds a saved `.R` script; the Console runs R commands. The Environment pane lists current objects, and the Plots and Help panes show figures and documentation. Saving a script keeps the instructions so that they can be read, changed and run again. Comments begin with `#`. In printed output, `[1]` marks the position of the first displayed element; it is not an answer or a command to type. The lab demonstrates the interface and file steps.

1.3 Values, vectors and function arguments

A vector contains values of one basic type. Scores are **numeric**; student names such as `"A"` are **character** values; a comparison returns **logical** values, `TRUE` or `FALSE`. Quotation marks distinguish text from an object name. `NA` denotes an unavailable value. Putting a word into a numeric vector, as in `c(50, "absent")`, converts it to character data; use `NA` for a missing score and retain its explanation separately.

```
scores + 5
#> [1] 55 65 75
scores[c(1, 3)]
#> [1] 50 70
scores > 55
#> [1] FALSE  TRUE  TRUE
scores[scores > 55]
#> [1] 60 70
```

The first operation adds five to each score, giving 55, 65 and 75; it does not replace `scores` unless the result is assigned. Positions start at 1, so the second line selects 50 and 70. The condition gives `FALSE`, `TRUE`, `TRUE`, and the final line keeps 60 and 70. These are different observations from the original three-score summary.

Arithmetic uses `+`, `-`, `*`, `/` and `^`. Parentheses specify order: `2 + 3 * 4` gives 14, whereas `(2 + 3) * 4` gives 20. This matters when translating a statistical formula into R. Name intermediate quantities while translating a formula; doing so makes the calculation easier to read.

An **R function argument** specifies a function's input or an option. This programming term is distinct from a **study argument**: the complete question-level reasoning for one question–answer pair. For example, `mean(scores)` and `mean(x = scores)` use the same input; `round(60.25, digits = 1)` requests one decimal place. Keep intermediate results unrounded

and round the reported answer. R is case-sensitive: `scores` and `Scores` are different names. Type `?mean` in the Console and read the help page's purpose, arguments and examples. We introduce the missing-value argument `na.rm` beside its first calculation below.

1.4 Base R and a small selection of tidyverse tools

Base R here means the functions supplied with an ordinary R installation, including its standard statistics packages. The **tidyverse** is a family of additional packages with related conventions. They work within the same R language. The course combines direct base R calculations with a small selection from three packages:

Tool	Role in this course
Base R	Arithmetic, vectors, tables, indexing and ordinary statistics
ggplot2	Graphics from data, mappings and layers
dplyr	Selected table operations, grouped summaries and joins
tidyr	Long/wide reshaping, including matching paired measurements

Installing a package makes it available on the computer. `library()` loads an installed package for the current session. The expression `dplyr::filter()` names a function and its package explicitly, even without loading the package's function names. The lab explains both forms. `ggplot2` is a package; `ggplot()` is one of its functions.

We show one small selection in both styles in Section 2, then use the clearest short expression for each statistical task. There is no requirement to implement every analysis in two styles or to load the entire tidyverse. Base R remains the main tool for the descriptive calculations.

1.5 Where Quarto fits

A script stores calculations. **Quarto** connects those calculations with prose, mathematics, tables, figures and references in a `.qmd` source document. It can render that source to PDF, HTML or Word. PDF is the main professional-report format here, with HTML as a useful second format. RStudio can edit the source and start a render; Quarto is a separate application. For R chunks it uses **knitr** to run code and insert results. PDF typesetting also needs a TeX installation, such as TinyTeX.

Section 7 shows a small report source. The lab collects one-time setup at the beginning and gives the optional production steps later. Understanding these roles belongs in the chapter; producing a complete report need not occupy the classroom statistics session.

2 Select Observations and Preserve the Measurement

2.1 Rows, columns and missing values

A data frame is a table whose columns may contain different types of values. Here each row is a student; the two score columns are measurements on that same student. Programme codes label categories, not quantities.

```
small <- data.frame(
  student = c("A", "B", "C"),
  program_code = c(1, 2, 2),
  baseline = c(50, 60, 70),
  followup = c(55, NA, 78)
)
small
#>   student program_code baseline followup
#> 1       A            1       50       55
#> 2       B            2       60       NA
#> 3       C            2       70       78
```

`NA` means a value is unavailable. It is not zero. `$` selects a column by name, so `small$baseline` is the vector of three baseline scores. The available follow-up mean uses two observations:

```
mean(small$followup, na.rm = TRUE)
#> [1] 66.5
sum(!is.na(small$followup))
#> [1] 2
```

The mean is 66.5 points and its denominator is two. `is.na()` identifies missing entries; `!` reverses a logical condition. `na.rm = TRUE` asks the function to exclude missing values from this calculation. It does not recover their values or make the included students representative of all three.

For change, subtract within the same student:

```
small$change <- small$followup - small$baseline
mean(small$change, na.rm = TRUE)
#> [1] 6.5
```

The changes are 5 and 8, giving 6.5 points for two complete pairs. In this particular table, subtracting the separate available-case means also gives 6.5, but the baseline mean uses three students. Numerical agreement does not establish matching denominators. A paired calculation should explicitly retain the same people for both measurements.

2.2 Inspect the observations needed for a summary

The built-in `airquality` data set records daily measurements in New York from May to September 1973. The dataset documentation defines ozone in parts per billion (ppb), wind in miles per hour and maximum daily temperature in degrees Fahrenheit. A row is a day, not a person.

```
aq <- airquality
head(aq)
#>   Ozone Solar.R Wind Temp Month Day
#> 1    41     190  7.4   67     5   1
#> 2    36     118  8.0   72     5   2
#> 3    12     149 12.6   74     5   3
#> 4    18     313 11.5   62     5   4
#> 5    NA      NA 14.3   56     5   5
#> 6    28      NA 14.9   66     5   6
names(aq)
#> [1] "Ozone"   "Solar.R" "Wind"    "Temp"    "Month"   "Day"
dim(aq)
#> [1] 153   6
str(aq)
#> 'data.frame':    153 obs. of  6 variables:
#>  $ Ozone  : int  41 36 12 18 NA 28 23 19 8 NA ...
#>  $ Solar.R: int  190 118 149 313 NA NA 299 99 19 194 ...
#>  $ Wind   : num  7.4 8 12.6 11.5 14.3 14.9 8.6 13.8 20.1 8.6 ...
#>  $ Temp   : int  67 72 74 62 56 66 65 59 61 69 ...
#>  $ Month  : int  5 5 5 5 5 5 5 5 5 5 ...
#>  $ Day    : int  1 2 3 4 5 6 7 8 9 10 ...
sum(is.na(aq$Ozone))
#> [1] 37
required_columns <- c("Ozone", "Solar.R", "Month", "Temp")
stopifnot(all(required_columns %in% names(aq)))
stopifnot(all(aq$Month >= 5 & aq$Month <= 9))
```

There are 153 rows and six columns, including 37 missing ozone values. `head()` shows the first rows, `names()` gives the column names, `dim()` gives the row and column counts, and `str()` displays the structure and column types. `summary(aq)` gives a first statistical overview. These complementary views help identify the measurements before calculation. The two `stopifnot()` checks stop execution if a required column is absent or if a month code falls outside the documented May–September range. Passing these checks shows only that two declared structural expectations hold; it does not prove that every value is accurate or that the record is suitable for the question. Keep an explicit set for the ozone question:

```
ozone_rows <- !is.na(aq$Ozone)
aq_observed <- aq[ozone_rows, ]
ozone <- aq_observed$Ozone
length(ozone)
#> [1] 116
100 * length(ozone) / nrow(aq)
#> [1] 75.82
```

In `table[rows, columns]`, leaving the column position blank retains all columns. The ozone mean will use 116 recorded values. The recorded-value percentage is $100(116/153) = 75.8\%$, using all days as its denominator. Different questions can require different denominators on the same table.

For two variables, select rows with both measurements. A missing value in an unrelated variable need not remove a row. For example:

```
both <- complete.cases(aq[c("Ozone", "Solar.R")])
sum(both)
#> [1] 111
```

This comparison has 111 complete days. State the variables required by a calculation before choosing its rows. Complete records are an inclusion rule, not evidence of random sampling or absence of selection bias.

2.3 One selection in base R and dplyr

Both styles must answer the same question using the same missing-value rule. To keep July days with recorded ozone:

```
july_rule <- aq$Month == 7 & !is.na(aq$Ozone)
july_base <- aq[july_rule, ]
july_tidy <- dplyr::filter(aq, Month == 7, !is.na(Ozone))
nrow(july_base)
#> [1] 26
nrow(july_tidy)
#> [1] 26
mean(july_tidy$Ozone)
#> [1] 59.12
```

Each result has the same 26 days and mean ozone 59.12 ppb. `==` tests equality, `&` requires both conditions, and `!` reverses the missingness test. Base indexing names columns through `aq$`; `filter()` reads the column names within its supplied table. Its comma-separated conditions must all hold.

The native pipe `|>` passes its left-hand result as the first argument of the function on the right. Thus `aq |> dplyr::filter(Month == 7, !is.na(Ozone))` expresses the same selection. A

short pipe can make a sequence readable, but a direct call is often enough. It differs from ggplot's +, which adds a graphical layer. Keep the statistical inclusion rule explicit whichever notation is used.

2.4 Transform units without changing the measurement

Convert maximum temperature to Celsius while retaining the original:

```
aq$temp_c <- (aq$Temp - 32) * 5 / 9
mean(aq$Temp)
#> [1] 77.88
mean(aq$temp_c)
#> [1] 25.49
sd(aq$Temp)
#> [1] 9.465
sd(aq$temp_c)
#> [1] 5.258
```

For $Y = a + bX$, the mean becomes $a + b\bar{x}$, the SD becomes $|b|s_X$ and the variance becomes $b^2 s_X^2$. Subtracting 32 shifts the location; multiplying by 5/9 changes the numerical scale. The same daily maximum temperatures have been expressed in different units. All 153 temperatures are recorded, so these means use more days than the ozone mean.

3 Describe a Distribution

3.1 Center and variation answer different questions

For observed values $x_1, \ldots, x_n$, the mean and sample variance are

$$\bar{x} = \frac{1}{n}\sum_i x_i, \qquad s^2 = \frac{\sum_i (x_i - \bar{x})^2}{n-1}, \qquad s = \sqrt{s^2}.$$

The mean locates the center. The squared deviations quantify dispersion; their sum is divided by $n-1$ in the sample-variance convention used by R. The deviations sum to zero, leaving $n-1$ freely varying deviations. Under independent sampling from a common finite-variance distribution, this denominator makes s^2 unbiased for the process variance. Using it on a fixed file does not establish that sampling model.

For 50, 60 and 70, the squared deviations sum to 200. Thus $s^2 = 100$ square points and $s = 10$ points. Variance has squared units; SD has the original units. If the intended descriptive variance divides by the full record count n, name that convention: here it is 200/3.

```r
mean(scores)
#> [1] 60
sum((scores - mean(scores))^2)
#> [1] 200
var(scores)
#> [1] 100
sd(scores)
#> [1] 10
```

The median is the middle ordered value, or the mean of the middle two when the count is even. It is less sensitive than the mean to a very large observation. Replacing 70 by 100 changes the mean to 70 while the median remains 60; the SD rises to about 26.46.

```r
changed_scores <- c(50, 60, 100)
mean(changed_scores)
#> [1] 70
median(changed_scores)
#> [1] 60
sd(changed_scores)
#> [1] 26.46
```

An extreme observation is a feature to investigate, not automatically an error to delete. Check its measurement and relevance before changing the analytic records. A distribution can have the same mean as another while having a different spread or shape.

3.2 Quartiles, IQR and the numerical convention

The first and third quartiles locate the lower and upper quarters of an ordered distribution. The interquartile range is $IQR = Q_3 - Q_1$ and describes the spread of its central half. The range uses the two most extreme values and is especially sensitive to them.

```r
mean(ozone)
#> [1] 42.13
median(ozone)
#> [1] 31.5
sd(ozone)
#> [1] 32.99
quantile(ozone, probs = c(0, .25, .5, .75, 1), type = 7)
#>     0%    25%    50%    75%   100%
#>   1.00  18.00  31.50  63.25 168.00
IQR(ozone, type = 7)
#> [1] 45.25
```

The 116 ozone values have mean 42.13 ppb, median 31.5 ppb and SD 32.99 ppb. Their minimum, first quartile, median, third quartile and maximum are 1, 18, 31.5, 63.25 and 168 ppb. The IQR is 45.25 ppb.

R's default `type = 7` uses interpolation. For sorted values, its position is $h = 1 + (n - 1)p$. At $p = .25$ for `c(1, 3, 3, 5)`, $h = 1.75$, giving $Q_1 = 1 + .75(3 - 1) = 2.5$. At $p = .75$, $Q_3 = 3 + .25(5 - 3) = 3.5$, so the IQR is 1. Chapter 2's medians-of-halves calculation gives an IQR of 2. State the convention when reproducing a small-sample answer.

3.3 Read the shape alongside the numbers

```
library(ggplot2)
ggplot(aq_observed, aes(x = Ozone)) +
  geom_histogram(binwidth = 10, color = "white") +
  labs(x = "Ozone (ppb)", y = "Days")
```

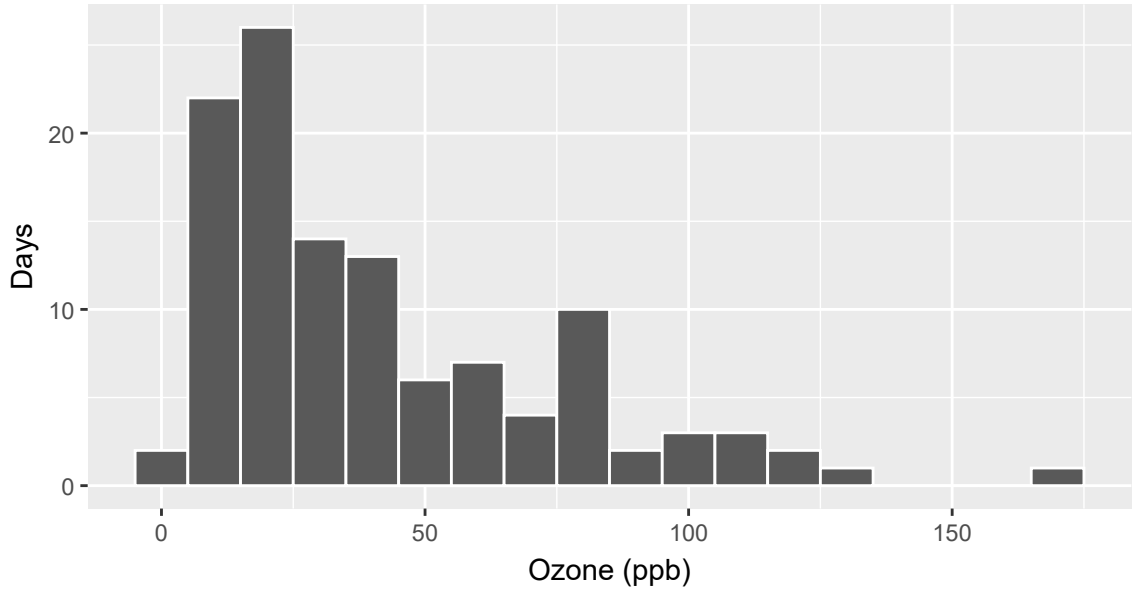

Figure 1: The distribution of the 116 recorded ozone values.

`ggplot()` identifies the data, `aes()` maps a variable to an axis, `geom_histogram()` groups values into bins and `labs()` supplies labels. The `+` adds a plot layer. The right tail helps explain why the mean exceeds the median. Changing bin width changes the display, not the observations or their arithmetic mean.

```r
ggplot(aq_observed, aes(x = Ozone, y = "Recorded days")) +
  geom_boxplot() +
  labs(x = "Ozone (ppb)", y = NULL)
```

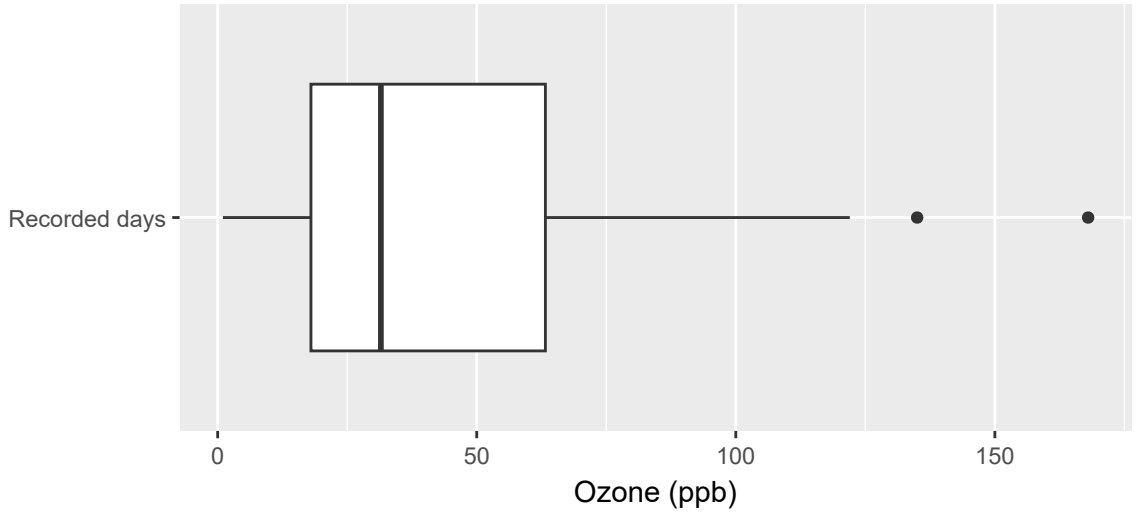

Figure 2: A boxplot summarizes the center, middle spread and unusually high ozone values.

The box displays quartiles and the median. Each whisker reaches the most distant observation no more than 1.5 times the IQR below Q_1 or above Q_3; values beyond those fences are drawn separately. These are display flags, not a decision to remove data. Small- sample boxplot conventions may differ from a hand quartile calculation. Always retain the units and inspect the values behind an unusual point.

4 Compare Groups and Conditional Percentages

4.1 Read the group count beside its mean

To compare recorded ozone across months, label the month codes and calculate each group's count, mean and SD. `factor()` marks a categorical variable; `tapply(values, groups, function)` applies one statistical function to each group. This short form keeps the variable and grouping explicit.

```
aq_observed$month <- factor(aq_observed$Month,
  levels = 5:9, labels = c("May", "June", "July", "August", "September"))
table(aq_observed$month)
#>
#>       May      June      July    August September
#>        26         9        26        26        29
tapply(ozone, aq_observed$month, mean)
#>       May      June      July    August September
#>     23.62     29.44     59.12     59.96     31.45
tapply(ozone, aq_observed$month, sd)
#>       May      June      July    August September
#>     22.22     18.21     31.64     39.68     24.14
```

The counts are 26, 9, 26, 26 and 29. Their means are approximately 23.62, 29.44, 59.12, 59.96 and 31.45 ppb. June's summary describes nine recorded days, not all 30 June days. A gap between the observed monthly means does not by itself isolate an effect of season or establish population uncertainty.

The same grouped operation can be made explicit with a short dplyr sequence:

```
monthly_ozone <- aq_observed |>
  dplyr::group_by(month) |>
  dplyr::summarise(
    n = dplyr::n(),
    mean_ozone = mean(Ozone),
    sd_ozone = sd(Ozone),
    .groups = "drop"
  )
monthly_ozone
#> # A tibble: 5 x 4
#>   month         n mean_ozone sd_ozone
#>   <fct>     <int>      <dbl>    <dbl>
#> 1 May          26       23.6     22.2
#> 2 June          9       29.4     18.2
#> 3 July         26       59.1     31.6
#> 4 August       26       60.0     39.7
#> 5 September    29       31.4     24.1
```

`group_by(month)` marks which rows belong together; it does not yet collapse the data. `summarise()` then returns one row per month, `n()` counts the rows contributing under the current ozone inclusion rule, and `.groups = "drop"` removes the grouping structure from the result. The new row therefore means a month summary, not an individual day, and its `n` is the relevant denominator.

```r
ggplot(aq_observed, aes(x = month, y = Ozone)) +
  geom_boxplot() +
  labs(x = "Month", y = "Ozone (ppb)")
```

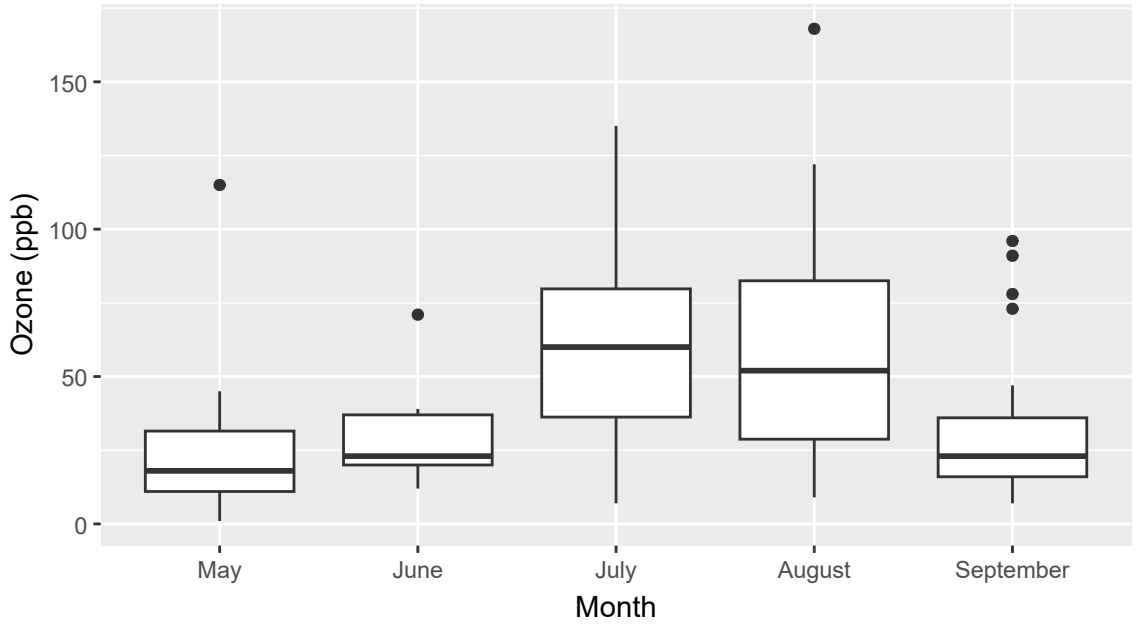

Figure 3: Recorded ozone distributions by month; each box uses that month's available measurements.

The overall mean of individual observations weights each group mean by its contributing count:

$$\bar{x} = \frac{\sum_g n_g \bar{x}_g}{\sum_g n_g}.$$

Two observations with mean 20 and eight with mean 50 have overall mean $(2 \times 20 + 8 \times 50)/10 = 44$. The unweighted mean of 20 and 50 is 35; it gives equal weight to groups. These summaries answer different questions.

4.2 Choose the denominator of a conditional percentage

The mtcars documentation describes 32 car models from 1973-74. `mpg` is miles per US gallon and `am` is automatic (0) or manual (1) transmission. Define 20 mpg as a teaching threshold and compare its frequency by transmission:

```
cars <- mtcars
cars$transmission <- factor(cars$am, levels = c(0, 1),
  labels = c("Automatic", "Manual"))
cars$mpg20 <- factor(cars$mpg >= 20, levels = c(FALSE, TRUE),
  labels = c("Below 20", "At least 20"))
tab <- table(cars$transmission, cars$mpg20)
tab
#>
#>             Below 20 At least 20
#>   Automatic       15           4
#>   Manual           3          10
```

There are 19 automatic models (15 below 20 and 4 at least 20) and 13 manual models (3 below 20 and 10 at least 20). All 32 have the needed measurements. A row percentage conditions on transmission:

```
100 * prop.table(tab, margin = 1)
#>
#>             Below 20 At least 20
#>   Automatic    78.95       21.05
#>   Manual       23.08       76.92
100 * prop.table(tab, margin = 2)
#>
#>             Below 20 At least 20
#>   Automatic    83.33       28.57
#>   Manual       16.67       71.43
```

Among automatic models, $100(4/19) = 21.05\%$ meet the threshold; among manual models, $100(10/13) = 76.92\%$ do. The manual-minus-automatic difference is 55.87 percentage points. Among the 14 models meeting the threshold, $100(10/14) = 71.43\%$ are manual. The latter uses a column denominator and answers a different question. The row and column labels explain the output from `margin = 1` and `margin = 2`.

These are observed associations among car models. Cars with different transmissions also differ in weight and other features. Chapter 6 adds a chi-square reference under a stated model; that reference does not identify the causal effect of changing a transmission.

5 Describe Quantitative Relationships

5.1 Covariance retains units; correlation standardizes them

For the same n complete pairs of numerical measurements,

$$s_{XY} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{n - 1}, \qquad r = \frac{s_{XY}}{s_X s_Y}.$$

Covariance averages the cross-products of centered observations with the sample denominator. Positive cross-products occur when the two measurements are on the same side of their respective means. Covariance has product units; correlation is unitless and lies between -1 and 1 when both SDs are positive.

For $x = (1, 2, 3)$ and $y = (2, 4, 6)$, the centered cross-products sum to 4, so covariance is $4/(3 - 1) = 2$. Their SDs are 1 and 2, giving $r = 1$.

```
pair_rows <- complete.cases(cars[c("wt", "mpg")])
car_pairs <- cars[pair_rows, ]
x <- car_pairs$wt
y <- car_pairs$mpg
length(x)
#> [1] 32
cov(x, y)
#> [1] -5.117
cor(x, y)
#> [1] -0.8677
```

Weight is recorded in thousands of pounds. The 32 pairs have covariance -5.1167 (thousand pounds times mpg) and correlation $-.8677$. This describes a strong negative linear association in these records.

```r
ggplot(car_pairs, aes(x = wt, y = mpg)) +
  geom_point() +
  labs(x = "Weight (thousand pounds)", y = "Fuel economy (mpg)")
```

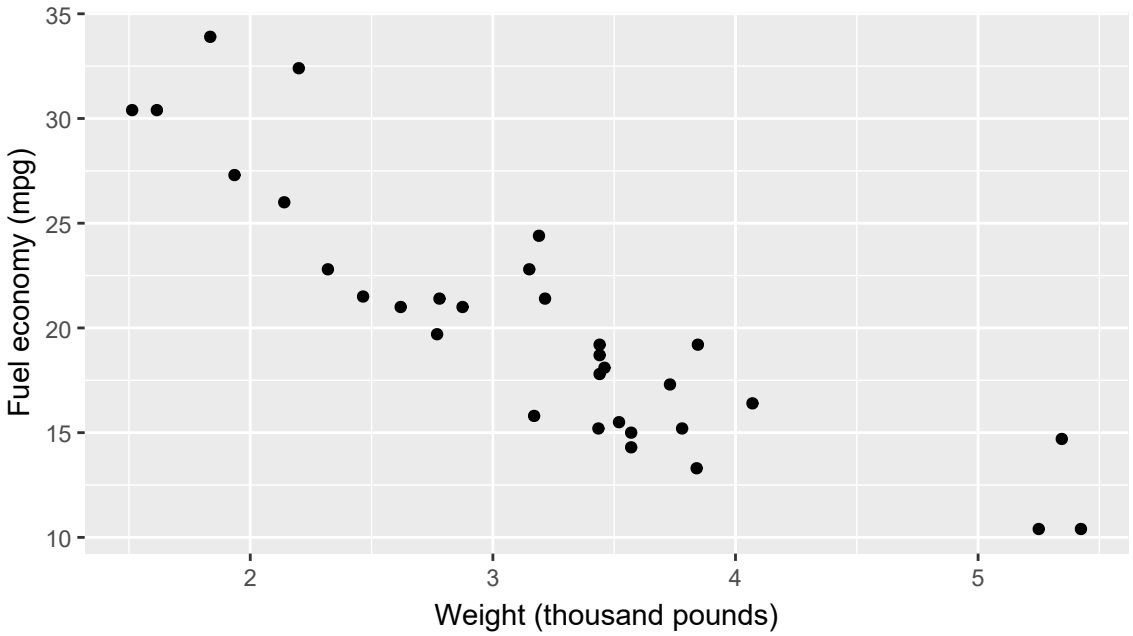

Figure 4: Weight and fuel economy for the same 32 car-model records used in the correlation.

Converting weight to pounds multiplies its covariance with mpg by 1000, but leaves the correlation unchanged. Adding a constant changes neither covariance nor correlation. Multiplying one variable by a negative constant reverses the correlation's sign. Correlation requires paired measurements, not independently sorted columns; use the same complete rows for the covariance and both SDs.

5.2 A small correlation can miss a strong curved pattern

Consider five fixed pairs on a curve:

```r
curve <- data.frame(x = c(-2, -1, 0, 1, 2))
curve$y <- curve$x^2
cov(curve$x, curve$y)
#> [1] 0
cor(curve$x, curve$y)
#> [1] 0
```

Both covariance and correlation are zero. Nevertheless, each y is exactly x^2. Positive and negative centered cross-products cancel by symmetry. The coefficient summarizes linear association; it does not summarize every possible relationship.

```
ggplot(curve, aes(x, y)) +
  geom_point() +
  geom_line() +
  labs(x = "X", y = "Y = X squared")
```

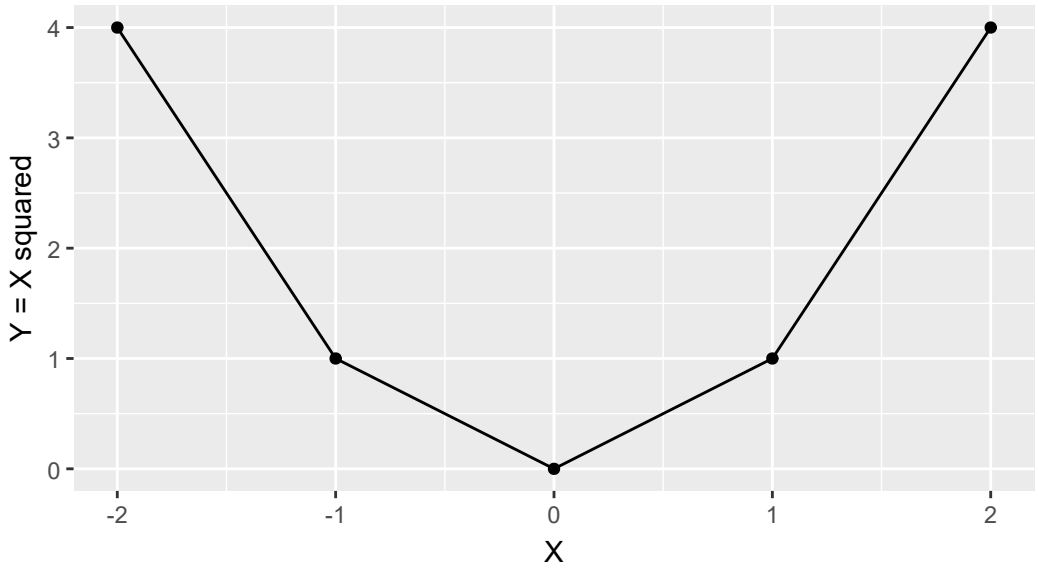

Figure 5: Five observations with zero Pearson correlation and an exact curved relationship.

Read form, direction, spread and unusual observations from the display, then interpret the numerical coefficient in that context. A correlation test in Chapter 6 addresses a model's zero linear-correlation question; it cannot recover a curved pattern discarded by the coefficient.

6 Describe Paired Differences

6.1 Align the measurements before subtracting

The sleep documentation describes extra hours of sleep relative to control under two drugs for ten people. `ID` identifies a person, `group` identifies the drug condition and `extra` is the measured extra sleep. One original row is a person-condition measurement. Twenty condition rows therefore represent ten people.

```
table(sleep$ID, sleep$group)
#>
#>      1 2
#>   1  1 1
#>   2  1 1
#>   3  1 1
#>   4  1 1
#>   5  1 1
#>   6  1 1
#>   7  1 1
#>   8  1 1
#>   9  1 1
#>   10 1 1
sleep_wide <- tidyr::pivot_wider(sleep, id_cols = ID,
  names_from = group, values_from = extra, names_prefix = "drug_")
sleep_wide$difference <- sleep_wide$drug_2 - sleep_wide$drug_1
head(sleep_wide, 3)
#> # A tibble: 3 x 4
#>   ID    drug_1 drug_2 difference
#>   <fct>  <dbl>  <dbl>      <dbl>
#> 1 1        0.7    1.9        1.2
#> 2 2       -1.6    0.8        2.4
#> 3 3       -0.2    1.1        1.3
```

`pivot_wider()` puts each ID's two measurements on one row. Its arguments name the person, condition and measurement. This operation preserves pairing even if the original rows are not ordered by condition. It does not resolve duplicate measurements for an ID-condition combination; those need review. The first difference is $1.9 - .7 = 1.2$ hours. Drug 2 minus drug 1 is the chosen direction throughout Chapters 5 and 6. These are not baseline and follow-up measurements, and negative `extra` means less sleep than control.

6.2 Describe the distribution of differences

```
complete_pair <- complete.cases(sleep_wide[c("drug_1", "drug_2")])
difference <- sleep_wide$difference[complete_pair]
length(difference)
#> [1] 10
mean(difference)
#> [1] 1.58
sd(difference)
#> [1] 1.23
range(difference)
#> [1] 0.0 4.6
sum(difference > 0)
#> [1] 9
```

All ten pairs are complete. Their differences sum to 15.8 hours, giving a mean of 1.58 hours and an SD of 1.23 hours. Nine are positive, one is zero and none is negative; the range is 0 to 4.6 hours. The mean describes the average difference and the SD describes variation among the differences.

When both condition means use the same people,

$$\overline{y_2 - y_1} = \bar{y}_2 - \bar{y}_1, \qquad s_D^2 = s_2^2 + s_1^2 - 2s_{12}.$$

The condition means are .75 and 2.33 hours, whose difference is 1.58. Their SDs are about 1.79 and 2.00, and their covariance is about 2.85. The positive within-person covariance reduces the variance of differences relative to the sum of the marginal variances. R can check the identity:

```
paired <- sleep_wide[complete_pair, ]
mean(paired$drug_2) - mean(paired$drug_1)
#> [1] 1.58
var(paired$drug_2) + var(paired$drug_1) -
  2 * cov(paired$drug_2, paired$drug_1)
#> [1] 1.513
var(difference)
#> [1] 1.513
```

The equality requires the same pairs for all terms. Subtracting two available-case means can compare different people when a measurement is missing. Correlation between conditions and the mean condition difference answers a different question: closely related measurements need not have equal means.

```r
ggplot(sleep_wide, aes(x = drug_1, y = drug_2)) +
  geom_point() +
  geom_abline(intercept = 0, slope = 1, linetype = 2) +
  labs(x = "Drug 1: extra sleep (hours)",
       y = "Drug 2: extra sleep (hours)")
```

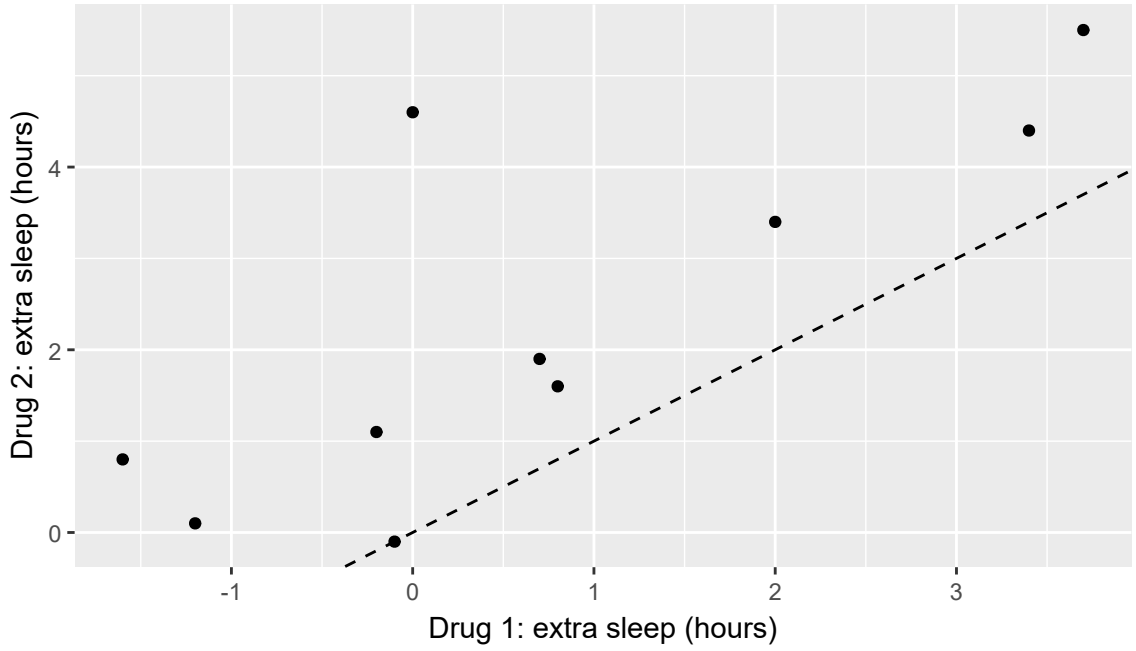

Figure 6: Extra sleep under two drugs for the same ten people; the diagonal marks equal condition values.

The diagonal is an equality reference, not a fitted regression line. An observed mean of 1.58 hours does not mean every person has that difference. Chapter 6 asks a further question about a process mean and introduces its SE and confidence interval under stated conditions.

7 Report the Descriptive Answer

7.1 Connect the number to the observations

A short descriptive report can combine center, spread and scope:

> Ozone was recorded on 116 of 153 days in the New York May-September 1973 dataset. Among those recorded days, mean ozone was 42.13 ppb, median 31.5 ppb,

> SD 32.99 ppb and IQR 45.25 ppb using R's type-7 quartiles. The distribution had a right tail. Monthly means were accompanied by their available counts; June contributed nine observed values.

Read the reported answer first. The data and inclusion rule identify the evidence, the definitions justify the arithmetic, and the calculations support a claim about the included days. A separate quality evaluation asks whether the definitions, records and scope fit the question. The report does not establish the ozone distribution for all unrecorded days, later years or other cities. Computing more decimal places would not resolve those representation questions.

Keep the commands producing a report in a short script. A useful report names the data, variables, units, missing-value rule, denominator and statistical convention. A Quarto document can connect those commands to prose and figures; the optional report and lab show the operating steps.

7.2 Combine a descriptive result into a Quarto report

If your instructor provides the optional classroom companion, obtain `quarto_example.qmd` and `quarto_example.bib` from the course space and save them beside each other. The source illustrates a report containing text, mathematics, R code, tables and graphics; the bibliography stores the reference records. Its data are built into R, so no separate data file is required.

A source begins with a YAML header. A compact version states its title, output formats and bibliography:

```
---
title: "Recorded ozone in New York"
format:
  pdf: default
  html: default
bibliography: quarto_example.bib
---
```

The source body uses ordinary Markdown and mathematical notation:

```
## Data and summary

We describe days with a recorded ozone measurement.
The observed-value mean is
$$
\bar{x} = \frac{1}{n}\sum_{i=1}^{n}x_i.
$$
```

An executable R chunk opens with three backticks followed immediately by `{r}`, and closes with three backticks. Place these option lines and commands between those fences:

```
#| label: tbl-ozone-summary
#| tbl-cap: "Observed ozone summary."
aq <- airquality
observed <- aq[!is.na(aq$Ozone), ]
n_observed <- nrow(observed)
mean_ozone <- mean(observed$Ozone)
knitr::kable(
  data.frame(n = n_observed, mean_ppb = mean_ozone),
  digits = 2
)
```

When rendered, the chunk produces 116 observations, a mean of 42.13 ppb, and a numbered table. In the source sentence, put the engine marker `r`, one space, and `round(mean_ozone, 2)` between single backticks to insert the computed mean. Write `Table @tbl-ozone-summary` to refer to the labelled table by its current number. A later chunk can draw the histogram with ggplot2 and use a `fig-` label in the same way. A citation key such as `[@peng2011]` connects a sentence to its bibliography entry.

The optional example declares an online APA citation style, so it needs no separate local style file. Its full header adds presentation settings. Rendering runs the calculations and produces the chosen document format. When a calculation changes, render again so that the numbers, table, figure and explanation can be read together. A report still needs a clear question, denominator, units and an appropriately limited answer. The lab gives the exact PDF and HTML commands and a short guided edit.

7.3 Summary and Reading Bridge

The statistical sequence is question, observations, summary, display and bounded interpretation. Simple R makes that sequence reproducible. Different summaries answer different questions: SD describes dispersion, conditional percentages depend on their denominator, correlation concerns linear form and a paired difference concerns measurements on the same unit.

Chapter 2 supplied these descriptive ideas. Chapter 6 adds sampling or working-model uncertainty to an explicitly stated estimand, using the same paired example and basic group comparisons. Chapter 7 connects covariance and variance to the regression slope; Chapter 8 requires consistent analytic records when comparing adjusted models. None of these steps makes a descriptive record summary automatically representative or causal.

Use local help such as `?sd`, `?quantile` and `?cor` for computational conventions, and R Core Team (2025) for R's reference system. The chapter explains the statistical reasons; the lab provides practice with the short commands.

Exercises

Use the stated records and conventions. Explain the statistical meaning as well as the calculation. Report numerical results to the requested precision.

Exercise 1: Count the right observations (Foundational)

A table contains 40 students with one baseline and one follow-up row each, plus one duplicate follow-up row. There are 81 rows. Explain the difference between a row count and a student count. If four students have missing follow-up scores, give the denominator for the mean observed follow-up score after the documented duplicate is resolved. State what the mean describes.

Exercise 2: Center, spread and a changed observation (Foundational)

Three recorded scores are 50, 60 and 70 points. Calculate the mean, median, sample variance and sample SD. Replace 70 by 100 and calculate the new mean, median and SD. Explain what changed statistically. Use the sample-variance denominator $n - 1$ and report SDs to two decimal places.

Exercise 3: A missing score is not zero (Foundational)

Students A, B and C have baseline scores 50, 60 and 70, and corresponding follow-up scores 55, missing and 78. Calculate the available follow-up mean, the complete-pair mean change and the difference of the separate available means. State each denominator. Explain why equality of two numerical answers would not establish that they used the same student sets.

Exercise 4: Choose the group denominator (Foundational)

A table classifies 19 automatic car models, of which 4 meet a fuel-economy threshold, and 13 manual models, of which 10 meet it. Calculate the percentage meeting the threshold within each transmission group and their manual-minus- automatic percentage-point difference. Then calculate the percentage of threshold-meeting models that are manual. Explain the different denominators.

Exercise 5: Read a relationship before evaluating it (Integrative)

For five paired observations, $x = (-2, -1, 0, 1, 2)$ and $y = (4, 1, 0, 1, 4)$. Calculate covariance and Pearson correlation and describe the scatterplot. A report says, "The correlation is zero, so the variables are unrelated." First identify its numerical result and reported conclusion, then evaluate whether that conclusion follows. What happens to covariance and correlation if y is multiplied by 10?

Exercise 6: Report paired differences (Integrative)

Four people have condition-1 values 1, 2, 4 and 5 and corresponding condition-2 values 2, 4, 4 and 8 hours. Calculate the mean and sample SD of condition 2 minus condition 1, and verify the difference-of-means identity. Write a descriptive result that names the people, direction and units. Explain why it does not say that every person gained the mean difference.

Appendix A: Supporting Table Operations

These optional operations support more complicated data layouts. The main statistical route requires only the short ID-based pivot already shown.

Return to a long layout when it is useful

For a display or calculation that uses one outcome column and a condition column, `pivot_longer()` reverses this change of layout:

```
sleep_long <- tidyr::pivot_longer(
  sleep_wide[c("ID", "drug_1", "drug_2")],
    cols = c(drug_1, drug_2),
    names_to = "drug",
    values_to = "extra"
  )
head(sleep_long)
#> # A tibble: 6 x 3
#>   ID    drug   extra
#>   <fct> <chr>  <dbl>
#> 1 1     drug_1   0.7
#> 2 1     drug_2   1.9
#> 3 2     drug_1  -1.6
#> 4 2     drug_2   0.8
#> 5 3     drug_1  -0.2
#> 6 3     drug_2   1.1
```

`cols` names the measurement columns to stack; `names_to` and `values_to` name the two resulting columns. Selecting the two drug columns first keeps the derived difference out of the stack: it is a comparison, not a third drug condition. The long result again has 20 measurement rows.

Add information from a lookup table

A **join** combines information using a shared identifier or **key**. Suppose programme labels are held in a separate lookup rather than written directly into a factor:

```
student_codes <- small[c("student", "program_code")]
program_lookup <- data.frame(
  program_code = c(1, 2),
  program = c("Standard", "Practice")
)
student_codes
#>   student program_code
#> 1       A            1
#> 2       B            2
#> 3       C            2
program_lookup
#>   program_code  program
#> 1            1 Standard
#> 2            2 Practice
labelled_students <- dplyr::left_join(
  student_codes, program_lookup, by = "program_code"
)
labelled_students
#>   student program_code  program
#> 1       A            1 Standard
#> 2       B            2 Practice
#> 3       C            2 Practice
```

The argument `by` names the shared key. A `left_join()` keeps the rows from the left table and adds matching information from the right. There are still three students. Students B and C both receive the Practice label because both have code 2.

This is a **many-to-one** relationship: many students may have one code, but each code should identify one row in the lookup. A simple count helps check that expectation:

```
table(program_lookup$program_code)
#>
#> 1 2
#> 1 1
```

Both codes occur once. An unmatched code in the student table would remain in a left join with a missing programme label. That differs from a repeated lookup key, which can multiply rows.

For example, suppose students A and B attend school S1 and student C attends S2. If a school lookup has two S1 rows and one S2 row, an unrestricted left join produces five rows: two for A,

two for B and one for C. It has not created two more students. Check the school lookup key and resolve what the repeated rows mean; taking the first three joined rows would arbitrarily lose information.

When this expectation is central to a task, `left_join()` can also express it with the argument `relationship = "many-to-one"`. It reports an error if a left-hand row matches more than one right-hand row. The declaration makes the expected relationship explicit; the key's meaning still comes from the study.

Files and a short reproducible report

A saved R script records the analytical steps. For a simple CSV, use `read.csv()` and inspect the imported columns before calculating. The student lab provides an optional read/write example and Quarto instructions. A report can combine prose, equations, code, tables and plots. Keep its variable definitions, inclusion rule and interpretation visible; a successful render is not evidence of an adequate statistical claim.

References

R Core Team. (2025). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. https://www.R-project.org/